HW4_Philbert_Fan

May 15, 2026

1 HW4: Palmer Penguins — Summary Statistics & Plots

1.0.1 Setup

- Use only `pandas`, `matplotlib`, and `seaborn`.
- Do your work in the cells provided; you may add extra cells.

[28]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# (Optional) nicer display for DataFrames
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 3)
```

[29]:
```
### Load the dataset
penguins = pd.read_csv("penguins.csv")

# Standardize column names (snake_case)
penguins.columns = [c.strip().lower().replace(' ', '_') for c in penguins.
 ↪columns]

penguins.head()
```

[29]:
```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0

   body_mass_g     sex  year
0       3750.0    male  2007
1       3800.0  female  2007
2       3250.0  female  2007
3          NaN     NaN  2007
4       3450.0  female  2007
```

1.0.2 Part 1) Data Vitals

Answer the following questions.

Your answer to each question must include some code that produces the result. You can, but you do not have to use a complete sentence to answer each question. Add cells as needed.

- a. What is the shape of the data?
- b. What are the Column names and data types of each column?
- c. How many values in each column are missing?
- d. How many penguins are there of each species? How many male and female penguins are there?

[30]:
```
print(penguins.shape)
print()
print(penguins.dtypes)
print()
print(penguins.isna().sum())
print()
print(penguins['species'].value_counts())
print()
print(penguins['sex'].value_counts())
```

```
(344, 8)

species               object
island                object
bill_length_mm       float64
bill_depth_mm        float64
flipper_length_mm    float64
body_mass_g          float64
sex                   object
year                   int64
dtype: object

species               0
island                0
bill_length_mm        2
bill_depth_mm         2
flipper_length_mm     2
body_mass_g           2
sex                  11
year                  0
dtype: int64

species
Adelie       152
Gentoo       124
Chinstrap     68
Name: count, dtype: int64
```

```
sex
male      168
female    165
Name: count, dtype: int64
```

a. The shape of the data is (344, 8). There are 8 columns are 344 rows or observations.

b. There are eight columns. The columns are 'species' which of the 'object' type, 'island' of the 'object' type, 'bill_length_mm' of the 'float64' type, 'bill_depth_mm' of the 'float64' type, 'flipper_length_mm' of the 'float64' type, 'body_mass_g' of the 'float64' type, 'sex' of the 'object' type, and 'year' of the 'int64' type.

c. There are a couple of columns with missing values. There are 2 missing values in 'bill_length_mm', 2 missing values in 'bill_depth_mm', 2 missing values in 'flipper_length_mm', 2 missing values in 'body_mass_g', and 11 missing values in 'sex'.

d. There are 152 penguins in the Adelie species, 124 in the gentoo species, and 68 in the chinstrap species. There are also 168 males and 165 female penguins in the dataset.

1.0.3 Part 2) Handle missing data

- a. Create a copy `penguins_clean` that **drops** rows with missing values in `['bill_length_mm','bill_depth_mm','flipper_length_mm','body_mass_g','sex']`.

- b. Report the new shape of `penguins_clean`.

[31]:
```
penguins_clean = penguins.copy()
penguins_clean = penguins_clean.dropna(subset=['bill_length_mm',␣
↪'bill_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex'])
print(penguins_clean.shape)
```

```
(333, 8)
```

The new shape of penguins_clean is 333 rows and 8 columns. 11 rows were dropped in total.

1.0.4 Part 3) Numeric summary statistics

- a. Compute the following **for each numeric column**: count, mean, std, min, 25%, 50%, 75%, max (hint: `.describe()`)
- b. Compute the mean and std for each numeric column for each `species`

[32]:
```
print(penguins_clean.describe())
print()
print("The Mean of Each Numeric Column for Each Species")
print(penguins_clean.groupby('species').mean(numeric_only=True))
print()
print("The Standard Deviation of Each Numeric Column for Each Species")
print(penguins_clean.groupby('species').std(numeric_only=True))
```

```
       bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g      year
count         333.000        333.000            333.000      333.000   333.000
```

```
mean           43.993         17.165            200.967     4207.057  2008.042
std             5.469          1.969             14.016      805.216     0.813
min            32.100         13.100            172.000     2700.000  2007.000
25%            39.500         15.600            190.000     3550.000  2007.000
50%            44.500         17.300            197.000     4050.000  2008.000
75%            48.600         18.700            213.000     4775.000  2009.000
max            59.600         21.500            231.000     6300.000  2009.000

The Mean of Each Numeric Column for Each Species
           bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  \
species
Adelie             38.824         18.347            190.103     3706.164
Chinstrap          48.834         18.421            195.824     3733.088
Gentoo             47.568         14.997            217.235     5092.437

               year
species
Adelie     2008.055
Chinstrap  2007.971
Gentoo     2008.067

The Standard Deviation of Each Numeric Column for Each Species
           bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g  \
species
Adelie              2.663          1.219              6.522      458.620
Chinstrap           3.339          1.135              7.132      384.335
Gentoo              3.106          0.986              6.585      501.476

            year
species
Adelie     0.812
Chinstrap  0.863
Gentoo     0.789
```

1.0.5 Part 4) Two-way grouped summaries

a) For each **(species, sex)** combination, compute the **mean** and **count** of:

- `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`.

b) Which (species, sex) has the **largest average body_mass_g**? Show the row.

[41]:
```
print(penguins_clean.groupby(['species', 'sex']).agg({
    'bill_length_mm': ['mean', 'count'],
    'bill_depth_mm': ['mean', 'count'],
    'flipper_length_mm': ['mean', 'count'],
    'body_mass_g': ['mean', 'count']}))
print()
```

```
print(penguins_clean.groupby(['species', 'sex'])['body_mass_g'].mean().
 ↪sort_values(ascending=False).head(1))
```

```
                 bill_length_mm       bill_depth_mm       flipper_length_mm  \
                           mean count          mean count              mean
species   sex
Adelie    female         37.258    73        17.622    73           187.795
          male           40.390    73        19.073    73           192.411
Chinstrap female         46.574    34        17.588    34           191.735
          male           51.094    34        19.253    34           199.912
Gentoo    female         45.564    58        14.238    58           212.707
          male           49.474    61        15.718    61           221.541

                      body_mass_g
                 count        mean count
species   sex
Adelie    female    73    3368.836    73
          male      73    4043.493    73
Chinstrap female    34    3527.206    34
          male      34    3938.971    34
Gentoo    female    58    4679.741    58
          male      61    5484.836    61

species  sex
Gentoo   male    5484.836
Name: body_mass_g, dtype: float64
```

1.0.6 Part 5) Correlations

- a. Compute the **correlation matrix** among numeric columns.
- b. Compute the correlation matrix **within each species** (hint: groupby).
- c. Find an example of Simpson's Paradox in the data. https://en.wikipedia.org/wiki/Simpson%27s_paradox
  - An example Simpson's Paradox would be a pair of variables that have a positive correlation within subgroups, but when all the groups are combined, there is negative correlation (or vice-versa).

[48]:
```
print(penguins_clean.corr(numeric_only=True))
print()
print(penguins_clean.groupby('species').corr(numeric_only=True))
```

```
                   bill_length_mm  bill_depth_mm  flipper_length_mm  \
bill_length_mm              1.000         -0.229              0.653
bill_depth_mm              -0.229          1.000             -0.578
flipper_length_mm           0.653         -0.578              1.000
body_mass_g                 0.589         -0.472              0.873
year                        0.033         -0.048              0.151
```

```
                   body_mass_g   year
bill_length_mm           0.589  0.033
bill_depth_mm           -0.472 -0.048
flipper_length_mm        0.873  0.151
body_mass_g              1.000  0.022
year                     0.022  1.000

                            bill_length_mm  bill_depth_mm  flipper_length_mm  \
species
Adelie    bill_length_mm             1.000          0.386              0.332
          bill_depth_mm              0.386          1.000              0.311
          flipper_length_mm          0.332          0.311              1.000
          body_mass_g                0.544          0.580              0.465
          year                       0.011         -0.235              0.328
Chinstrap bill_length_mm             1.000          0.654              0.472
          bill_depth_mm              0.654          1.000              0.580
          flipper_length_mm          0.472          0.580              1.000
          body_mass_g                0.514          0.604              0.642
          year                       0.042         -0.059              0.346
Gentoo    bill_length_mm             1.000          0.654              0.664
          bill_depth_mm              0.654          1.000              0.711
          flipper_length_mm          0.664          0.711              1.000
          body_mass_g                0.667          0.723              0.711
          year                       0.205          0.243              0.201

                            body_mass_g   year
species
Adelie    bill_length_mm          0.544  0.011
          bill_depth_mm           0.580 -0.235
          flipper_length_mm       0.465  0.328
          body_mass_g             1.000 -0.046
          year                   -0.046  1.000
Chinstrap bill_length_mm          0.514  0.042
          bill_depth_mm           0.604 -0.059
          flipper_length_mm       0.642  0.346
          body_mass_g             1.000  0.037
          year                    0.037  1.000
Gentoo    bill_length_mm          0.667  0.205
          bill_depth_mm           0.723  0.243
          flipper_length_mm       0.711  0.201
          body_mass_g             1.000  0.052
          year                    0.052  1.000
```

From the results, I can see that when all the groups are combined 'bill_depth_mm' has a negative correlation with all other columns. However, 'bill_depth_mm' has a positive correlation with 'bill_length_mm' when in subgroups. So an example of Simpson's Paradox is the correlation between 'bill_depth_mm' and 'bill_length_mm'.

1.1 Plots

- Each figure must have a **title**, **axis labels**, and a **legend** (when appropriate).
- Use readable tick labels and sensible limits.
- Be sure each graphic is displayed.

1.1.1 Part 6) Histograms of body mass by species

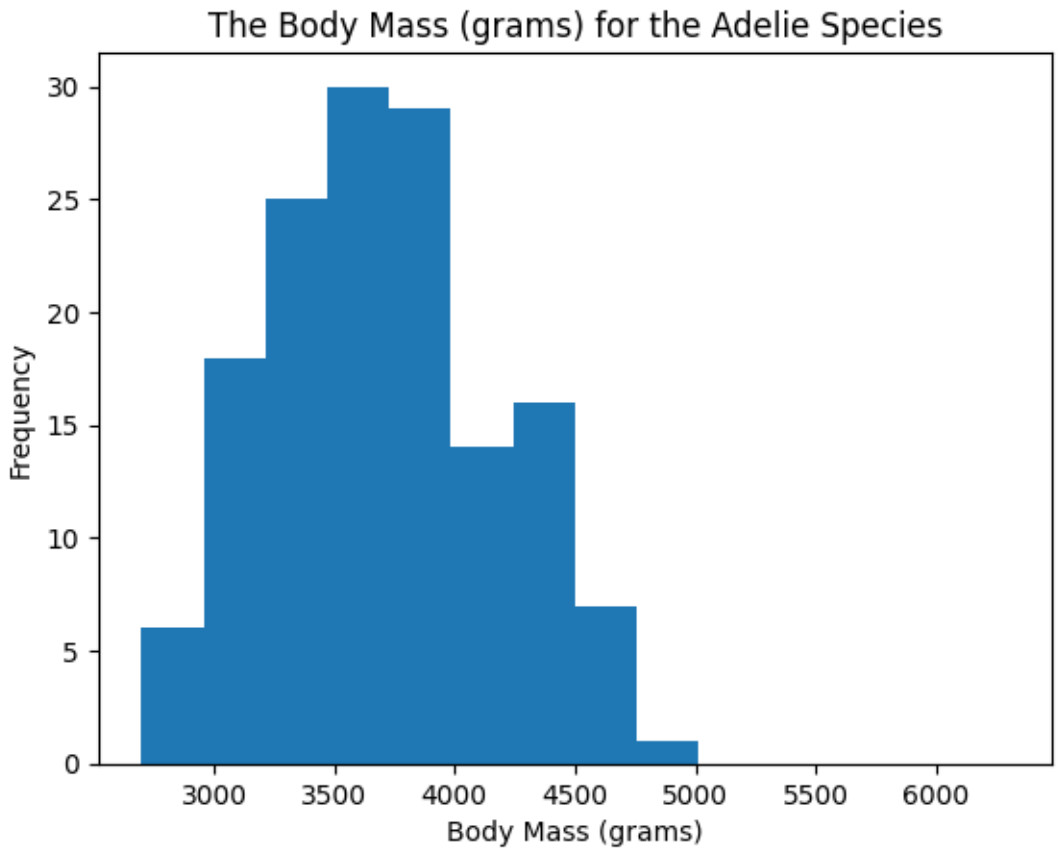
Create **separate** histograms of `body_mass_g` for each species (three panels).

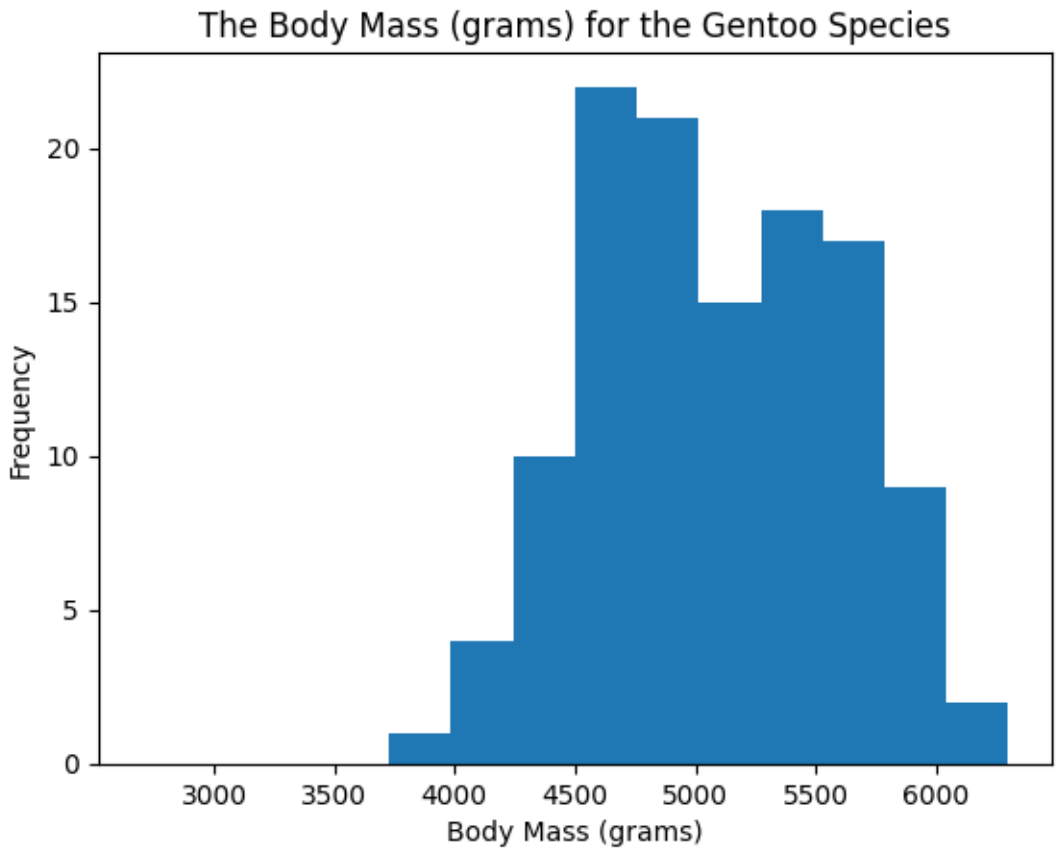
- Use the same binning across panels so comparisons are fair.

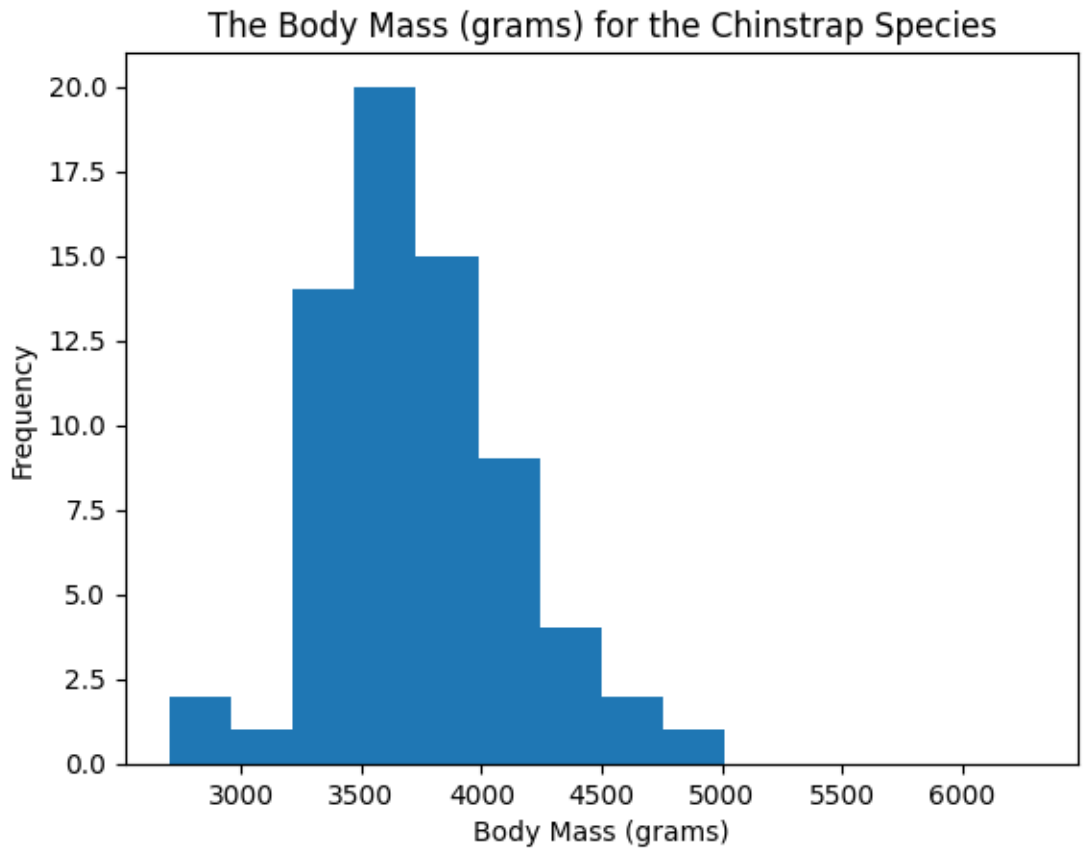
- Label axes clearly

[56]:
```
min_mass = penguins_clean['body_mass_g'].min()
max_mass = penguins_clean['body_mass_g'].max()
bins = np.linspace(min_mass, max_mass, 15)

for species in penguins_clean['species'].unique():
    plt.hist(penguins_clean[penguins_clean['species'] ==␣
 ↪species]['body_mass_g'], bins=bins)
    plt.title(f"The Body Mass (grams) for the {species} Species")
    plt.xlabel("Body Mass (grams)")
    plt.ylabel("Frequency")
    plt.show()
```


The Body Mass (grams) for the Adelie Species
Frequency
0
5
10
15
20
25
30
3000
3500
4000
4500
5000
5500
6000
Body Mass (grams)


The Body Mass (grams) for the Gentoo Species
Frequency
0
5
10
15
20
3000
3500
4000
4500
5000
5500
6000
Body Mass (grams)

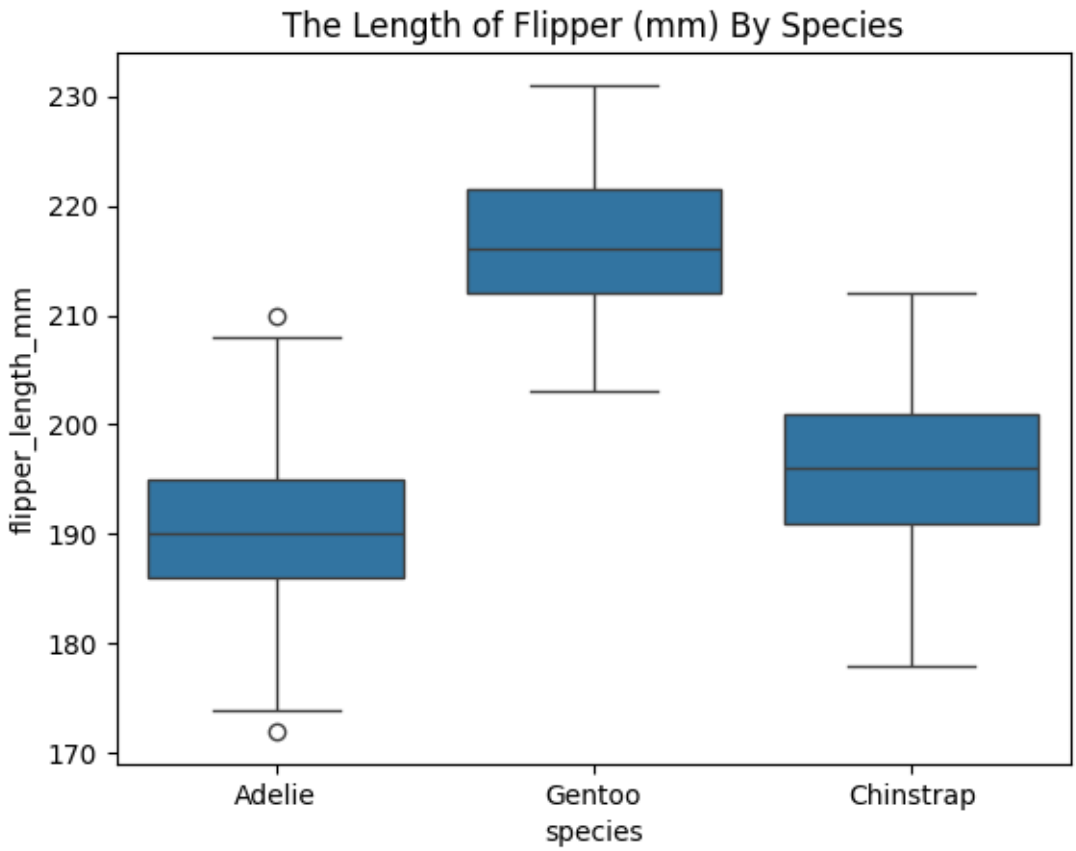


1.1.2 Part 7) Boxplots of flipper length

Make a **single figure** with boxplots of `flipper_length_mm` by **species**.

Briefly compare the medians and spreads in 2–3 sentences in a markdown cell.

[58]:
```
sns.boxplot(x=penguins_clean['species'], y=penguins_clean['flipper_length_mm'])
plt.title("The Length of Flipper (mm) By Species")
plt.show()
```

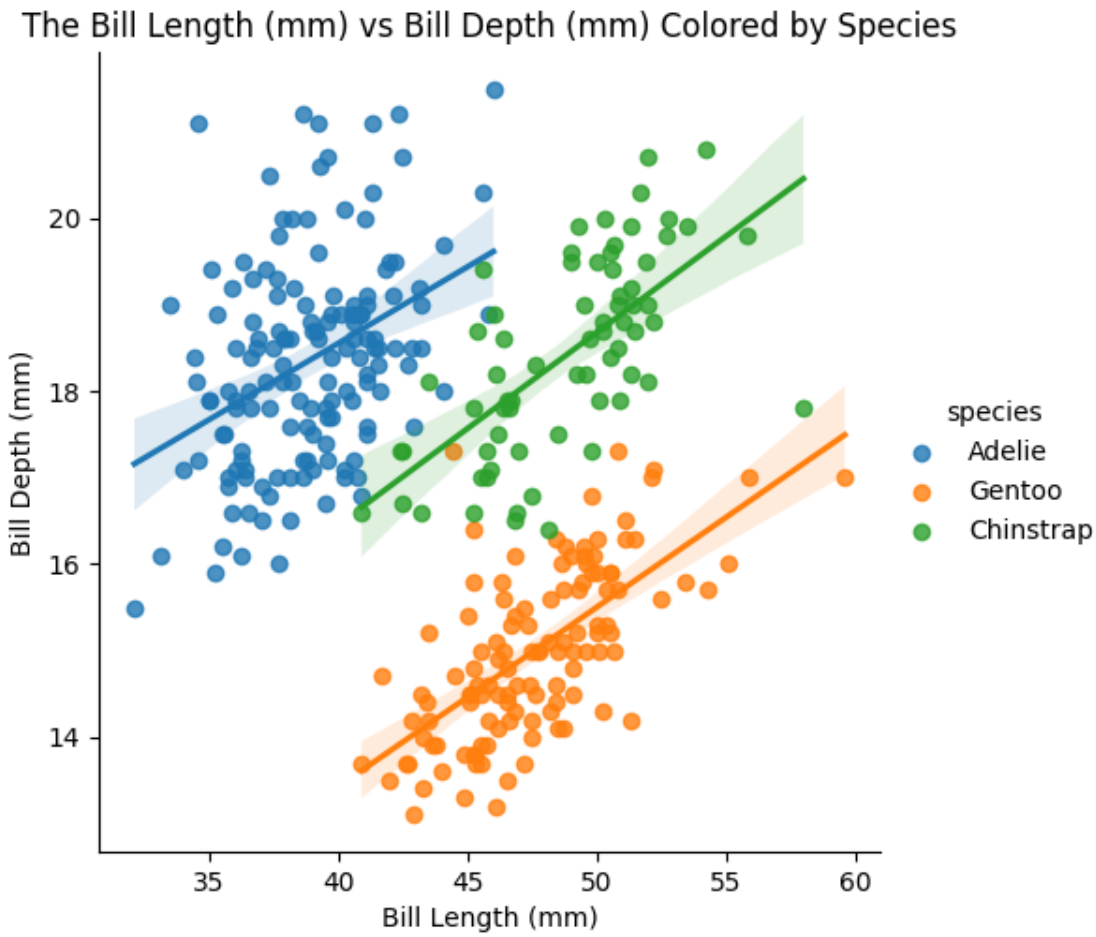


From the boxplots, the medians from each species looks very different. The Adelie species has the lowest median at 190mm and the highest spread, it also contains a couple of outliers as well. The Gentoo species has the lowest spread but the highest medianat around 216mm and no outliers. The Chinstrap species has a spread that is in between the other two species and the median around 197mm.

1.1.3 Part 8) Scatter: bill length vs bill depth

- Make a scatter plot of **bill_length_mm** (x) vs **bill_depth_mm** (y), colored by species.
- Add a simple **linear fit line** per species
- Include a legend

[66]:
```
sns.lmplot(data=penguins_clean, x='bill_length_mm', y='bill_depth_mm',␣
↪hue='species', legend=True)
plt.title('The Bill Length (mm) vs Bill Depth (mm) Colored by Species')
plt.xlabel('Bill Length (mm)')
plt.ylabel('Bill Depth (mm)')
plt.show()
```

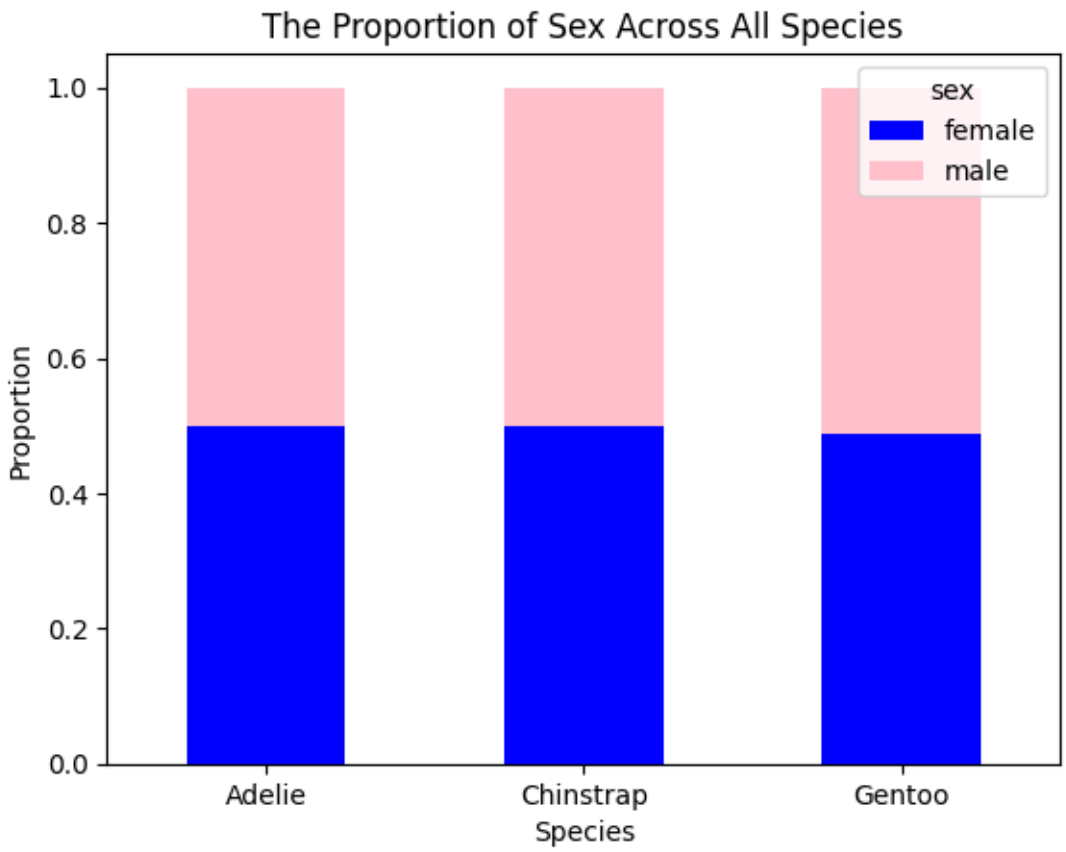


1.1.4 Part 9) Stacked bars: sex proportions by species

Compute counts of `sex` within each `species`, convert to **proportions**, and make a **stacked bar chart**.

- Bars should be species on the x-axis with male/female proportions stacked to 1.0.

[78]:
```
sex_count_within_species = penguins_clean.groupby('species')['sex'].
↪value_counts(normalize=True).unstack()
print(sex_count_within_species)
sex_count_within_species.plot(kind='bar', stacked=True, color=['blue', 'pink'],␣
↪legend = True)
plt.title("The Proportion of Sex Across All Species")
plt.xlabel("Species")
plt.xticks(rotation=0)
plt.ylabel("Proportion")
plt.show()
```

```
sex        female   male
```

```
species
Adelie      0.500  0.500
Chinstrap   0.500  0.500
Gentoo      0.487  0.513
```